

Entwurfsmuster

Rezepte für wiederkehrende Probleme

Warum Entwurfsmuster nutzen?

- Lösung bzw. "Rezepte" von häufig auftretenden Entwurfsproblemen.
- Nutzung bewährter Lösungen durch Erfahrung anderer Entwickler.
- Förderung gemeinsamer Sprache und effektiveren Kommunikation im Team.
- Nutzung standardisierter und wiederverwendbaren Komponenten.

Wie geht es euch?

Kocht ihr gerne? Mit oder ohne Rezept?

>> Anekdote in der ersten Wohnung Schinkennudeln kochen.

Im Lauf der Jahre lernte ich immer mehr Patterns kennen.

Irgendwann stößt man auf das Buch der **GoF**.

Buchempfehlung 1

- Für **Java** geschrieben, aber eignet sich trotzdem auch für andere OOP wie C#
- Leicht verständlich und unterhaltsame Einführung für Anfänger und Fortgeschrittene
- Interaktiver Lernansatz mit Rätsel, Quizze und Übungen
- Trotz des unterhaltsamen Stils bietet es tiefgehende Erklärungen



<https://www.oreilly.com/library/view/head-first-design/9781492077992/>

Es ist pädagogisch sehr fundiert mit vielen Übungen, netten Methaphern und beabsichtigter Redundanz.

Man könnte es auch als Lehrbuch für Lösungsfindung mit Praxisbezug verstehen, was es so besonders macht!

Leseprobe auf Amazon

<https://www.amazon.de/Entwurfsmuster-von-Kopf-bis-objektorientierte/dp/3960091621?dib=eyJ2IjoiMSJ9.g2DXOjGioiyAHoXUyGW6rCXaasS-IfdaYHV2-F0MLihKgsQaxFrxmpg3-EC7DNcb9uebNhUq1opnezxolPHAGfQvq8Bh1glhUjbYYDAuEcR7eW4o0mSj-FdJJvyO7YQ4EWJ0wzi11SKEeO5N-t2KK60cWbPdaHD6>

Buchempfehlung 2

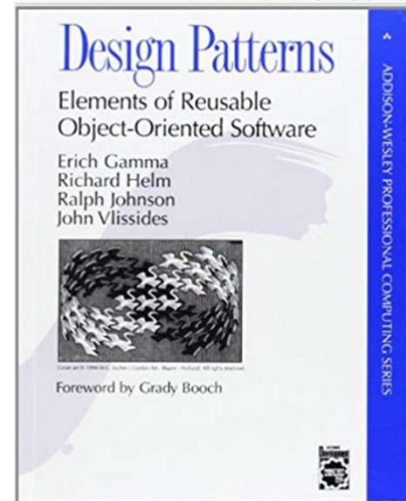
- Standardwerk der "Gang of Four" (GoF), welches Design Pattern populär machte
- Beschreibt 23 grundlegende Pattern, die in der Entwicklung weit verbreitet sind
- Richtet sich an erfahrene Entwickler und bietet tiefgehende Analyse der Muster
- Viele **Anregungen** sich mit OOP-Design auseinanderzusetzen
- Alternativen im Web



Refactoring Guru

dofactory

For C#



Erich Gamma: Sitzt heute bei Microsoft, federführend für VS Code.

1. Katalog von Entwurfsmustern:

- 23 klassische Entwurfsmuster in 3 Kategorien gegliedert
- Creational:
 - Wie werden Objekte erzeugt?
 - Singleton, Builder, FactoryMethod
- Structural:
 - Wie werden Objekte verbunden und integriert?
 - Wie können größere Strukturen aussehen?
 - Adapter, Decorator, Facade
- Behavioral Patterns
 - Wie verhalten sich Objekte und Objektstrukturen?
 - Observer, Iterator, Strategy

2. Detaillierte Beschreibungen

- Name und Intention / Motivation,
- Struktur, Beteiligte und Zusammenwirken,
- Vor- und Nachteile (Konsequenzen)
- Problembeschreibung und beispielhafte Lösung (vorgeschlagene Referenzimplementierung).

3. Prinzipien und Konzepte: Das Buch behandelt auch grundlegende Prinzipien und Konzepte der objektorientierten Programmierung, die den Entwurfsmustern zugrunde liegen.

Alle Patterns haben gemein, dass sie nach loser Kopplung streben.

Aber sie sind konkreter und greifbarer, als die reine Empfehlung "nach loser Kopplung" zu streben.

Stärken:

1.Grundlegendes Werk: Es gilt als eines der grundlegenden Werke im Bereich der Softwareentwicklung und hat viele nachfolgende Bücher und Ressourcen beeinflusst.

2.Tiefgehende Analyse: Die detaillierten Analysen und Erklärungen der Muster bieten ein tiefes Verständnis ihrer Anwendung und Implementierung.

3.Breite Anwendbarkeit: Die Muster sind weitgehend anwendbar und haben sich in vielen verschiedenen Programmiersprachen und Anwendungsbereichen bewährt.

Entwurfsmuster (Designpatterns)

- Creational patterns

- Singleton
- Factory
- Builder
- Prototyp

- Structural patterns

- Adapter
- Facade
- Composite
- Decorator
- Proxy

- Behavioral patterns

- Observer
- Template Method
- Strategy
- Iterator
- Command
- Memento
- State
- Visitor

[Refactoring Guru \(Catalog\)](#)

- Erzeugungsmuster
- Strukturmuster
- Verhaltensmuster

Disclaimer: Negative Seiten der Muster

- **Selbstzweck:** Patterns sind **kein** Allheilmittel und nicht für jedes Problem geeignet. Unkritische Anwendung kann zu Over-Engineering führen.
- **Lernaufwand:** Verständnis und korrekte Anwendung erfordern Zeit und Übung.
- Kann Komplexität des Systems erhöhen, insbesondere bei nicht korrekter Implementierung oder unnötiger Verwendung.
- Falsch angewendet und zu viel Abstraktion beeinträchtigen Lesbarkeit und Wartbarkeit des Codes.
- Patterns entstanden aus einer rein OOP-orientierten Perspektive und sollten daher nicht auf andere Paradigmen übertragen werden!
- Siehe auch Anti-Patterns: <https://de.wikipedia.org/wiki/Anti-Pattern>

Design-Patterns? Bloß nicht!

<https://youtu.be/oC-9HZ0r3e4>

1. Patterns als Selbstzweck (Cargo-Cult)

- Code wird nicht dadurch besser, dass er viele Patterns enthält
- Guter Code zeichnet sich durch Les- und Nachvollziehbarkeit aus
- Abstraktion kann das Gegenteil bewirken, weil man gedanklich auf mehreren Ebenen unterwegs sein muss.
Gerade dann, wenn Patterns **PRO**aktiv statt **RE**aktiv eingesetzt werden! (YAGNI)
Beispiel Factory: Create statt new schafft zusätzliche In Direktion, was schwieriger zu verstehen ist.
- Namensgebung kann leiden: Statt fachlicher Name ein technisch abstrakter Name
Verwässern die Lesbarkeit und verschleiern den fachlichen Zweck des Codes
- **Wir sollten uns gut überlegen, ob der Nutzen die Kosten übersteigt**

2. OOP

- Patterns entstanden aus OOP-orientierten Perspektive (siehe Untertitel des Buchs)
- Zeigt sich an verwendeter UML-Notation und Implementierungen in C++

- Sind **keine** universelle Wahrheiten, aber **existieren nur im OOP-Kontext**,
d. h. nominalem, statischen Typsystem
- Folge: strukturelle, dynamische Typsysteme machen Patterns überflüssig
- **Provokativ ausgedrückt:** Patterns lösen die Unzulänglichkeiten von OOP-Sprachen!

1. Entwickler "vergessen" zwischen **Intention** und **Implementierung** zu unterscheiden.

- D. h. Patterns sind keine strikten Rezepte.
- Häufig wird zu sehr an Beispielimplementierung als Referenz festgehalten!
- Besser ein Schritt zurück treten und fragen, was die Intention des Patterns war.
- Singleton in C# schwieriger als man glaubt
 - double-check locking algorithm
 - <https://jonskeet.uk/csharp/singleton.html>
 - <https://www.sudhanshutheone.com/posts/double-check-lock-csharp>

2. Die falschen Probleme lösen

- Beispiel Singleton: OOP-Variante einer globalen Variable.
- Schlechte Idee weil hohe Abhängigkeit wird große Probleme schaffen, insbesondere bei Tests.
- Besser: Zentrale Instanz einmalig erzeugen und als Parameter (DI) weitergeben.
- Vorteil: keine Abhängigkeiten nach außen, was Testbarkeit verbessert.

3. **ABER:** Manche Patterns sind so populär, dass sie es als Sprachfeatures geschafft haben

- Z. B. Iterator und Observer (Events, Promises, Observables)

Factory Method

Creational Pattern

<https://de.wikipedia.org/wiki/Fabrikmethode>

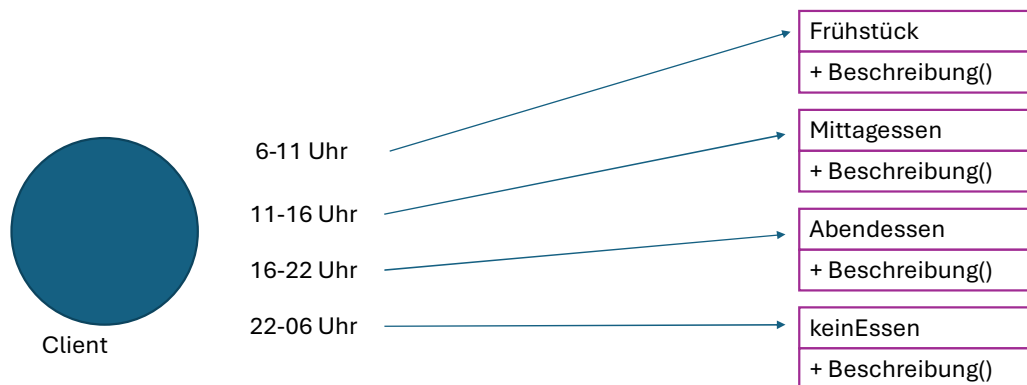
NUR DEMO IN VISUAL STUDIO!

- Factory (Pizza Store)
- Builder (Custom Pizza)
- Decorator (Slice, Boxing, Extra Cheese)
- Adapter (NY Style Pan Pizza)
- Strategy (Delivery)

Verwendung

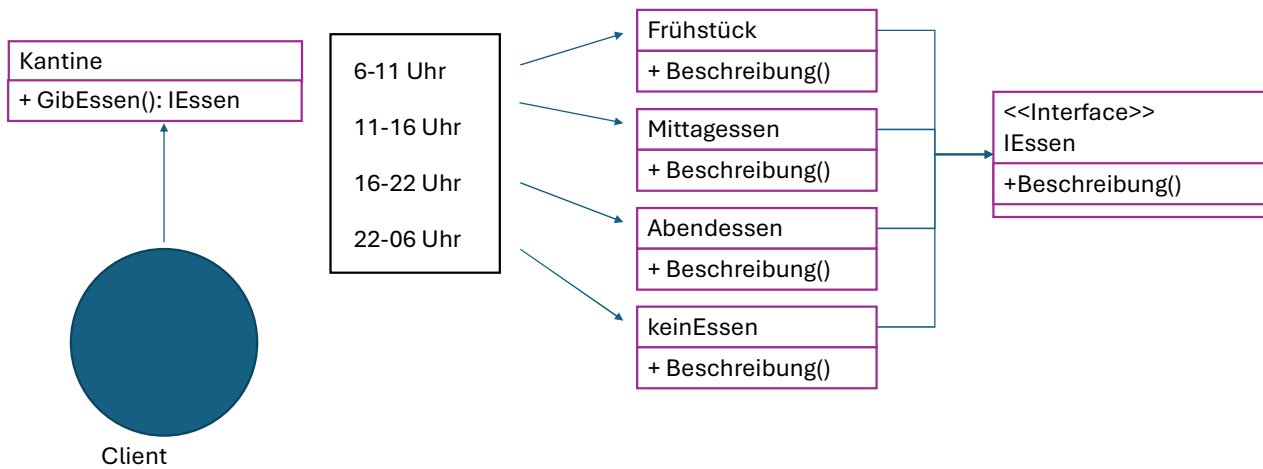
- Client soll sich nicht um die Objekterstellung kümmern, wenn verschiedene Möglichkeiten zur Verfügung stehen.
- Er soll nicht die Auswahl treffen, welche Implementierung genau instanziiert wird..
- Beispiel
 - Einlesen von Dateien in verschiedenen Formaten, ohne die Details zu kennen,
 - Object-Pooling
 - Threadpool
 - Connectionpool

Beispiel ohne Factory



- Der Client erzeugt die Mahlzeiten direkt basierend auf der Tageszeit.
- Es gibt keine zentrale Stelle, die die Erzeugung der Mahlzeiten verwaltet.

Beispiel mit Factory



- Eine zentrale Klasse (Kantine) mit einer Methode `GibEssen()` wird verwendet, um die Mahlzeiten zu erzeugen.
- Die Kantine verwendet das Interface `IEssen`.

Builder

Creational Pattern

[https://de.wikipedia.org/wiki/Erbauer_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Erbauer_(Entwurfsmuster))

Verwendung

- Komplex zu erzeugende Objekte.
- Viele (optionale) Parameter und Bestandteile.
- Mehrere Einzelschritte für die Erzeugung notwendig.
- Nicht alle Kombinationsmöglichkeiten sind auch zulässig.

Beispiel

- Bestellsystem für Wunschanfertigung von Schränken
 - Viele Wahlmöglichkeiten
 - Einigen obligatorisch, einige optional
 - Einige schließen sich gegenseitig aus.

Schrank

+ AnzahlTüren	→	Min. 2 – Max. 7
+ Oberfläche	→	Unbehandelt / Gewachst / Lackiert
+ Farbe	→	10 verschiedene Farben
+ Metallschienen	→	Bool -> false == Holz auf Holz / true == Aufpreis
+ AnzahlBöden	→	Min. 0 – Max. 6
+ Kleiderstange	→	Bool

Möglichkeiten ohne Builder

• Teleskopkonstruktoren

```
0 references | 0 changes | 0 authors, 0 changes
public Schrank() : this(2, 2, Schrankoberfläche.Unbehandelt, "", false, false)
{ }

0 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren) : this(anzahlTüren, 2, Schrankoberfläche.Unbehandelt, "", false, false)
{ }

0 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren, int anzahlBöden) : this(anzahlTüren, anzahlBöden, Schrankoberfläche.Unbehandelt, "", false, false)
{ }

0 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren, int anzahlBöden, Schrankoberfläche oberfläche) : this(anzahlTüren, anzahlBöden, oberfläche, "", false, false)
{ }

0 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren, int anzahlBöden, Schrankoberfläche oberfläche, string farbe) : this(anzahlTüren, anzahlBöden, oberfläche, farbe, false, false)
{ }

0 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren, int anzahlBöden, Schrankoberfläche oberfläche, string farbe, bool metallschienen) : this(anzahlTüren, anzahlBöden, oberfläche, farbe, metallschienen, false)
{ }

6 references | 0 changes | 0 authors, 0 changes
public Schrank(int anzahlTüren, int anzahlBöden, Schrankoberfläche oberfläche, string farbe, bool metallschienen, bool kleiderstange)
{
    AnzahlTüren = anzahlTüren;
    AnzahlBöden = anzahlBöden;
    Oberfläche = oberfläche;
    Farbe = farbe;
    Metallschienen = metallschienen;
    Kleiderstange = kleiderstange;
}
```


Builder

Schrank

- Schrank(anzahlTüren)

- + AnzahlTüren
- + Oberfläche
- + Farbe
- + Metallschienen
- + AnzahlBöden
- + Kleiderstange

Erbauer

- + Erbauer(anzahlTüren)
- + MitOberfläche(oberfläche)
- + InFarbe(farbe)
- + MitMetallschienen()
- + MitBöden(anzahlBöden)
- + MitKleiderstange()
- + Konstruiere()

```
Schrank meinSchrank = new Schrank.SchrankBuilder()  
    .SetBöden(2)  
    .SetTüren(4)  
    .Create();
```

Singleton

Creational Pattern

[https://de.wikipedia.org/wiki/Singleton_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Singleton_(Entwurfsmuster))

NUR DEMO IN VISUAL STUDIO!

Verwendung

- Instanziert maximal 1 Instanz der Klasse.
- Beispiele
 - Zentraler Cache
 - Zentrale Verwaltung externe Ressourcen (zb. Logdateien, programmweite Konfigurationsdaten)
 - Effiziente Zurverfügungstellung von readonly Daten
- Alternative
 - programmweite Konfiguration als Parameter den Konsumenten "durchreichen"

Singleton
- instance: Singleton
- Singleton() + Instance() : Singleton

Warum nicht einfach „static“?

-> Singelton kann Intefaces implementieren! Static nicht!

Threadsicherheit: Standardmäßig ist weder „static“ noch „Singelton“ threadsicher.

Adapter

Structural Pattern

[https://de.wikipedia.org/wiki/Adapter_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Adapter_(Entwurfsmuster))

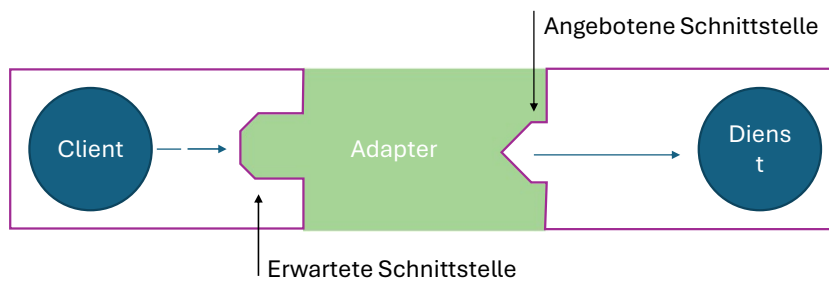
Verwendung

- Die Schnittstelle eines Objekts entspricht nicht der, die ein Client erwartet.
- Keines der beiden Objekte darf/kann verändert werden.

Beispiele

- Handy Ladekabel -> USB passt nicht in 230V Steckdose
- Vorhandene Bibliothek wird durch andere ersetzt, welche zwar die gleichen Dienste anbietet, aber mit anderen Klassen.
- Client soll mit Altsystem funktionieren / kommunizieren, welches aber bald ausgetauscht wird.

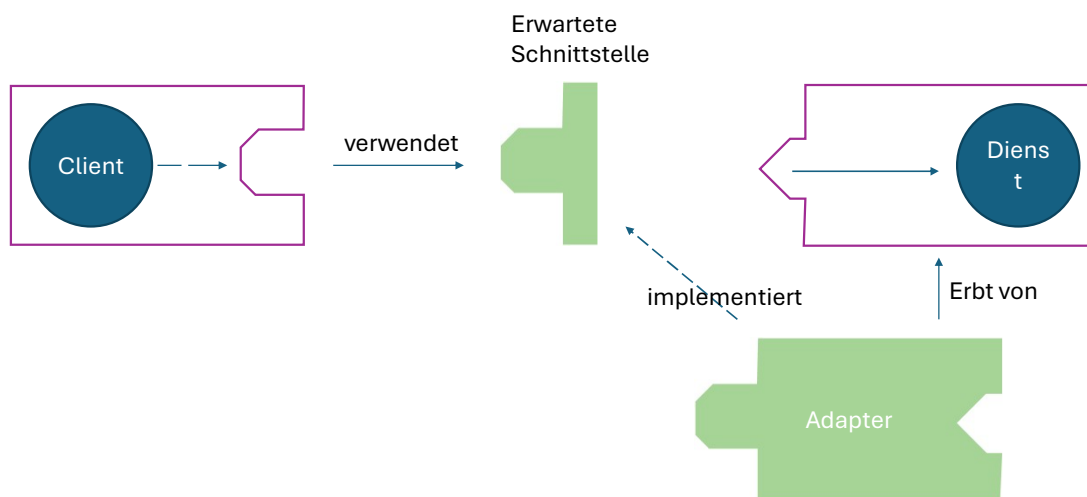
Adapter



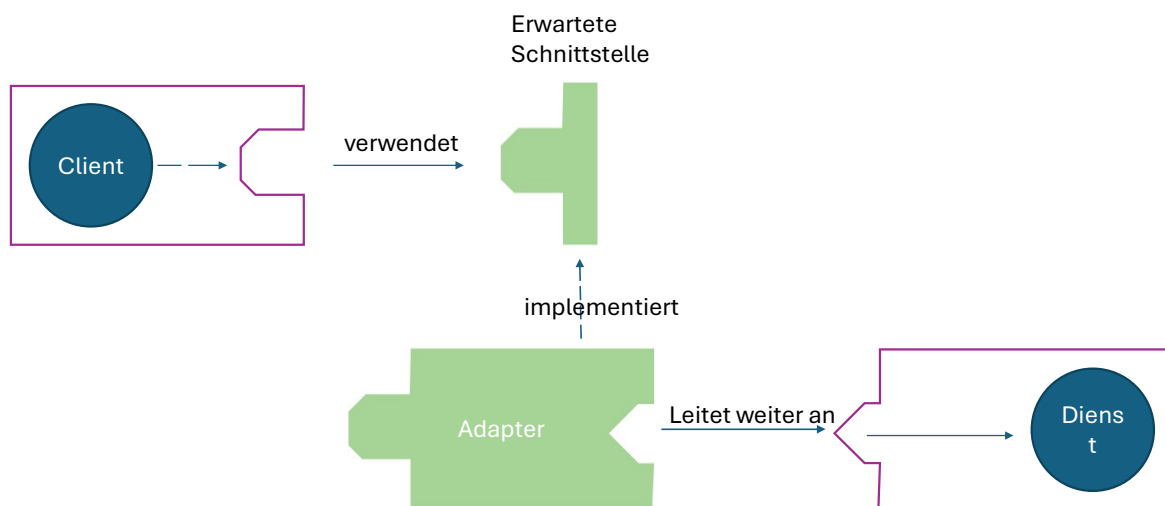
Zwei Möglichkeiten:

- Vererbung
- Komposition

Vererbung: Klassenadapter



Komposition: Objektadapter



Vergleich

- Vererbung:
 - Verbindung mit Dienst ist statisch
 - Kann genau einen Dienst/Klasse anpassen
- Komposition
 - Auch mit Unterklassen vom Dienst geeignet
 - Zur Laufzeit mit Dienst verknüpfbar

Vorteile

- Existierende Klasse ohne Änderung verwendbar, obwohl die Signatur nicht passt.
- Auf Objekt kann von verschiedenen Clients über unterschiedliche Schnittstellen zugegriffen werden.

Nachteile

- Aufwand kann minimal sein – aber auch beträchtlich!
- Zusätzliche Indirektion -> Overhead

Facade

Struktural Pattern

[https://de.wikipedia.org/wiki/Fassade_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Fassade_(Entwurfsmuster))

Verwendung / Problem

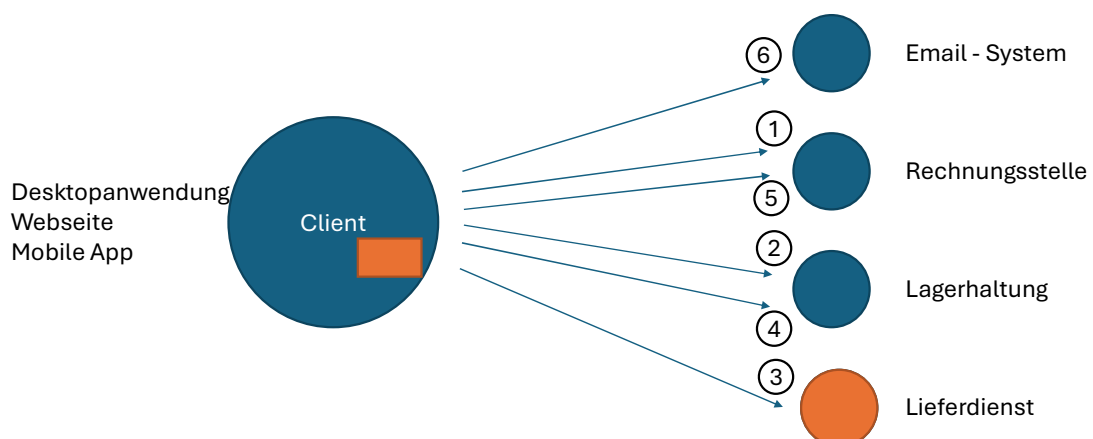
- Komplexes System oder Teilsystem verlangt viel Wissen vom Client über die Struktur und den internen Ablauf.
- Änderungen an einem System verlangen auch Änderungen am Client.

Beispiel

- Shopsystem

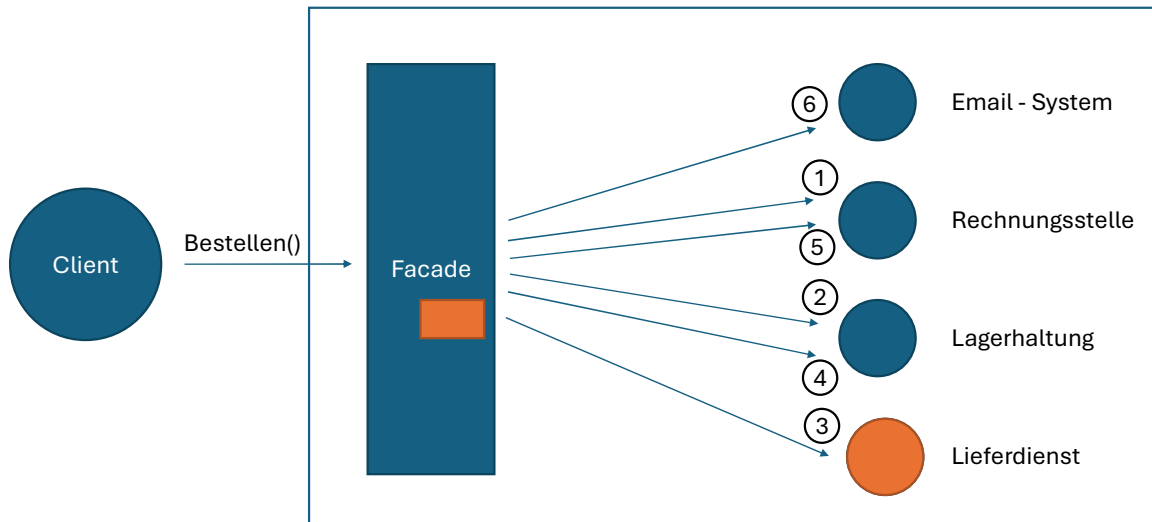
- Bestellung besteht aus vielen Einzelschritten
 - Reihenfolge der Schritte ist wichtig
-
1. Rechnungsstelle: Offene Rechnungen?
 2. Lagerhaltung: Produkt lagernd?
 3. Lieferdienst: Versand beauftragen.
 4. Lagerhaltung: Lager auffüllen.
 5. Rechnungsstelle: Bezahlvorgang starten.
 6. Email-System: Bestätigung schicken.

Problem



Bei jeder Änderung im System muss sich auch der Client ändern.

Lösung



Bei einer Änderung im System ändert sich nur die Fassade, nicht aber der Client.

Vorteile

- Client und Teilsysteme sind lose gekoppelt
- Client benötigt kein Wissen über die Teilsysteme
- Änderungen an einem Teilsystem haben keine Auswirkung auf den/die Clients

Nachteile

- Zusätzliche Indirektion
- Gefahr des Gott-Objekts
 - Fassade ist nur Koordinator!
 - eventuell Unterfassaden einführen

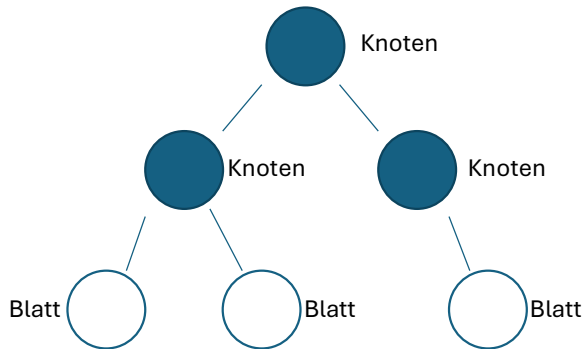
Composite

Structural Pattern

[https://de.wikipedia.org/wiki/Kompositum_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Kompositum_(Entwurfsmuster))

Verwendung

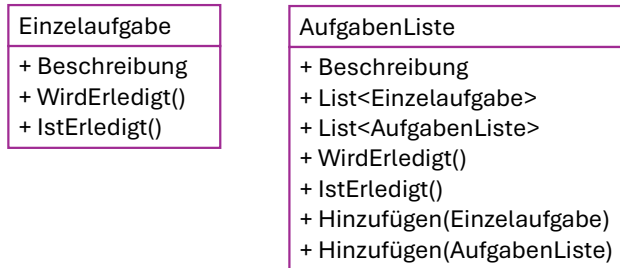
- Aufbau einer Baumstruktur



Beispiele:

- Grafische Oberflächen
- Abbildung von Verzeichnisstrukturen
- Hierarchische Aufgabenliste

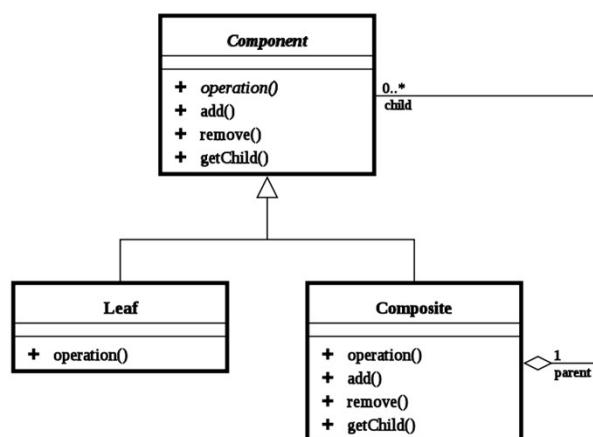
Ohne Composite



Probleme

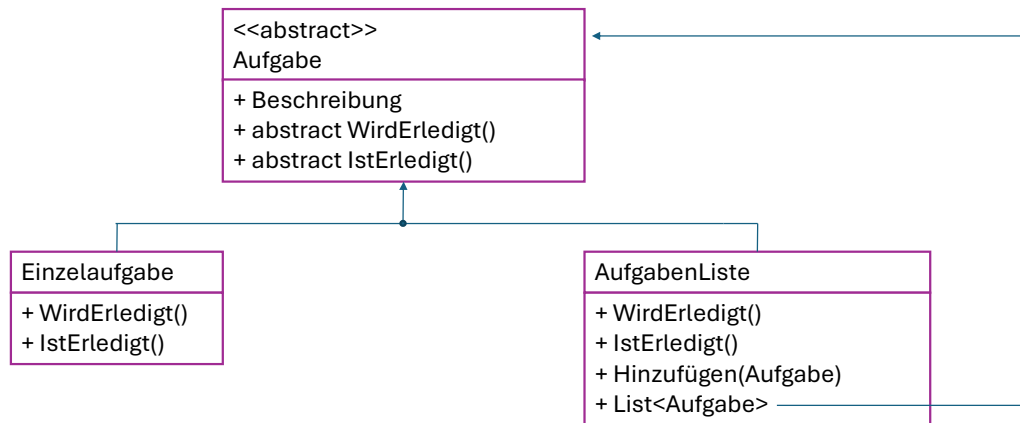
- Unterschiedliche Schnittstellen für Einzelaufgaben und Aufgabenlisten.
- Der Client muss wissen, ob er mit einer Einzelaufgabe oder einer Aufgabenliste arbeitet.
- Komplexität bei der Verwaltung und Erweiterung des Systems.

UML



Quelle: Wikipedia

Lösung mit Composite



- Einheitliche Schnittstelle für Einzelaufgaben und Aufgabenlisten.
- Der Client kann mit Einzelaufgaben und Aufgabenlisten auf die gleiche Weise interagieren.
- Erleichtert die Verwaltung und Erweiterung des Systems.
- Flexibilität bei der Erstellung komplexer Hierarchien von Aufgaben.

Ohne Composite:

Unterschiedliche Schnittstellen für Einzelaufgaben und Aufgabenlisten führen zu Komplexität und schwerer Wartbarkeit.

Mit Composite:

Eine einheitliche Schnittstelle für alle Aufgabenarten ermöglicht eine einfachere und flexiblere Verwaltung und Erweiterung des Systems. Der Client kann mit allen Aufgaben auf die gleiche Weise interagieren, unabhängig davon, ob es sich um Einzelaufgaben oder Aufgabenlisten handelt.

Vorteile

- Einheitliche Schnittstelle für Einzelaufgaben und Aufgabenlisten.
- Der Client kann mit Einzelaufgaben und Aufgabenlisten auf die gleiche Weise interagieren.
- Erleichtert die Verwaltung und Erweiterung des Systems.
- Flexibilität bei der Erstellung komplexer Hierarchien von Aufgaben.

Nachteile

- Spezielle Eigenschaften der Bauelemente nur erschwert möglich.
- Bauelemente bieten einzelne geerbte Methoden möglicherweise nicht an.

Decorator

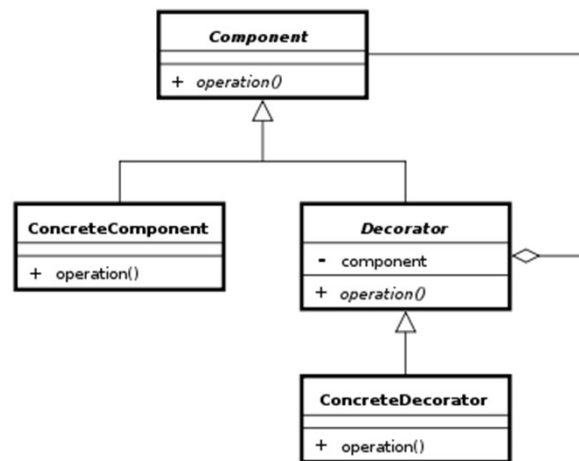
Structural Pattern

<https://de.wikipedia.org/wiki/Decorator>

Verwendung

- Dynamisch Funktionalität zu einem Objekt hinzuzufügen.
- Beispiel:
 - Streaming Klassen in C# & Java

UML



Quelle: Wikipedia

Vorteile

- Die Funktionalität kann ohne direkte Vererbung erweitert werden.
- Die Dekorierer können dynamisch ausgetauscht werden.

Nachteile

- Viele kleine ähnliche Klassen.
- Gefahr beim Testen auf Objektidentität.

Mediator

Behavioral Pattern

Verwendung

- Vereinfacht komplexe Systeme mit vielen interagierenden Komponenten
- Zentralisiert Kommunikation zwischen Objekten
- Reduziert direkte Verbindungen zwischen Objekten
- Entkoppelt interagierende Objekte

Beispiele

- Chatraum

- Chater melden sich an und ab
- Kommunizieren nicht direkt miteinander, sondern über den Chatraum
- Verschiedene Räume ermöglichen unterschiedliche Kommunikationsarten.
- Der Chatraum verwaltet Benutzer, Nachrichten und Räume zentral
- Leitet Nachrichten an die richtigen Empfänger weiter



- Fluglotse als Vermittler zwischen Flugzeugen

- Flugzeuge kommunizieren nicht direkt miteinander
- Der Fluglotse koordiniert den Luftraum und die Landebahnen
- Er gibt Anweisungen zu Höhe, Geschwindigkeit und Richtung
- Überwacht Starts und Landungen
- Löst potenzielle Konflikte zwischen Flugzeugen



Nuget Package MediatR

- IRequest / IReponse
- INotification



Vorteile

- Fördert lose Kopplung zwischen Objekten
- Erhöht Wiederverwendbarkeit von Objekten
- Vereinfacht Wartung und Erweiterung des Systems
- Zentralisiert Kontrolle und erleichtert Fehlerbehebung

Nachteile

- Mediator kann komplex werden und schwer zu warten sein
- Möglicher Flaschenhals für die Leistung
- Risiko eines Single Point of Failure
- Kann zu übermäßiger Zentralisierung führen

Nacheinander informiert -> können sich gegenseitig blocken.

Observer

Behavioral Pattern

[https://de.wikipedia.org/wiki/Beobachter_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Beobachter_(Entwurfsmuster))

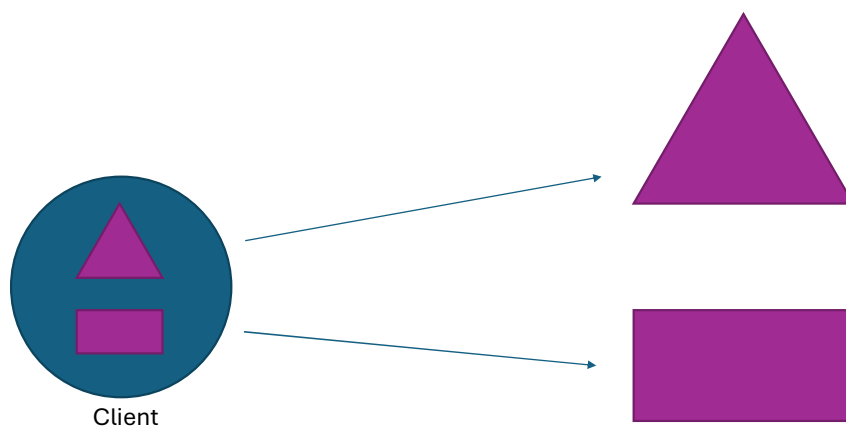
Problemstellung

- Wenn ein Objekt seinen Zustand ändert, sollen andere Objekte darüber informiert werden.

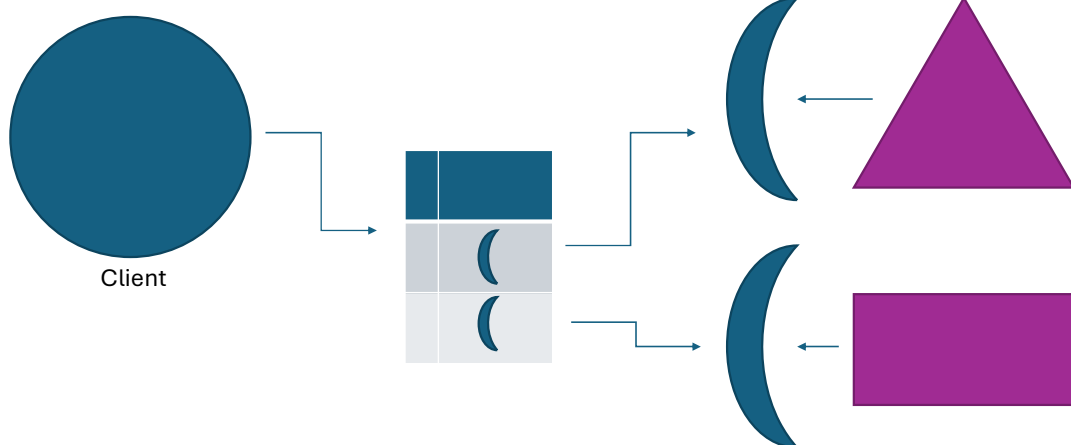
Beispiel

- Sensor liefert regelmäßig aktuelle Temperaturdaten.
- Heizung und Kühlung sollen dementsprechend ein- bzw. ausgeschaltet werden.

Problem



Lösung



Vorteile

- Entkopplung von Beobachtetem und Beobachtern
- Automatische Benachrichtigungen.
- Art und Anzahl der Beobachter ist nicht eingeschränkt.
- Beobachter können zur Laufzeit an- & abgemeldet werden.

Nachteile

- Zusätzliche Schnittstelle
- Kontrollfluss schwer nachvollziehbar
- Vermischung von technischem und fachlichem Code im Beobachteten
- Beobachter werden nacheinander informiert

Nacheinander informiert -> können sich gegenseitig blocken.

Template Method

Behavioral Pattern

<https://de.wikipedia.org/wiki/Schablonenmethode>

Verwendung

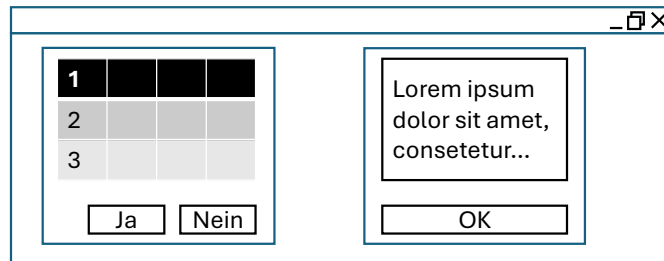
- Festgeschriebener Algorithmus aber Details können sich ändern.
- Reihenfolge wichtig.

Beispiele

- Sortieralgorithmus
 - Details: aufsteigend/absteigend
- Netzwerkverbindung
 - Verbindung aufbauen
 - Daten empfangen
 - Verbindung abbauen
 - Details: http/smtp

Beispiel: UI

- Rahmen zeichnen
- Inhalt zeichnen (Hintergrundbild/Text)
- Unterelemente zeichnen (Tabelle/Button)



Vorteile

- Zentrale Definition des Algorithmus
- Einzelne Schritte in Unterklassen anpassbar
- Grundstruktur des Algorithmus unveränderlich
- In Unterklassen auch optionale Erweiterungen möglich („hooks“)

Nachteile

- Grundstruktur des Algorithmus unveränderlich
- Kontrollfluss schwer nachvollziehbar
- große Anzahl zu überschreibender Methoden schlecht handhabbar

Strategie

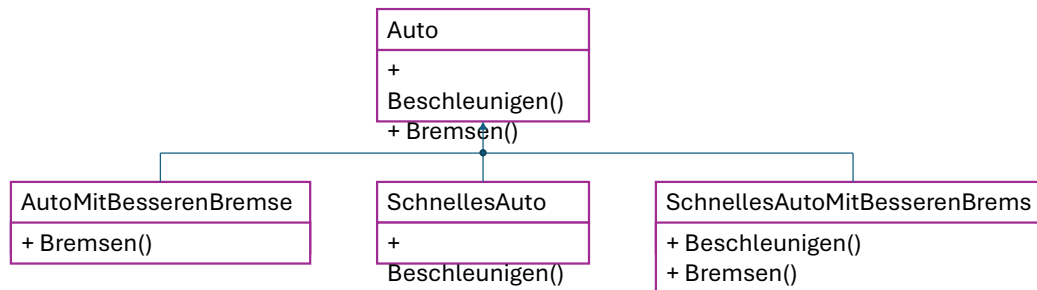
Behavioral Pattern

[https://de.wikipedia.org/wiki/Strategie_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Strategie_(Entwurfsmuster))

Verwendung

- Das Verhalten eines Objekts soll während der Laufzeit ausgetauscht werden.
- Beispiele:
 - Aus verschiedenen Sortieralgorithmen auswählen
 - Speichern einer Datei in verschiedenen Formaten
 - Auto Rennspiel: Beschleunigungs- & Bremsverhalten kann geändert/verbessert werden.

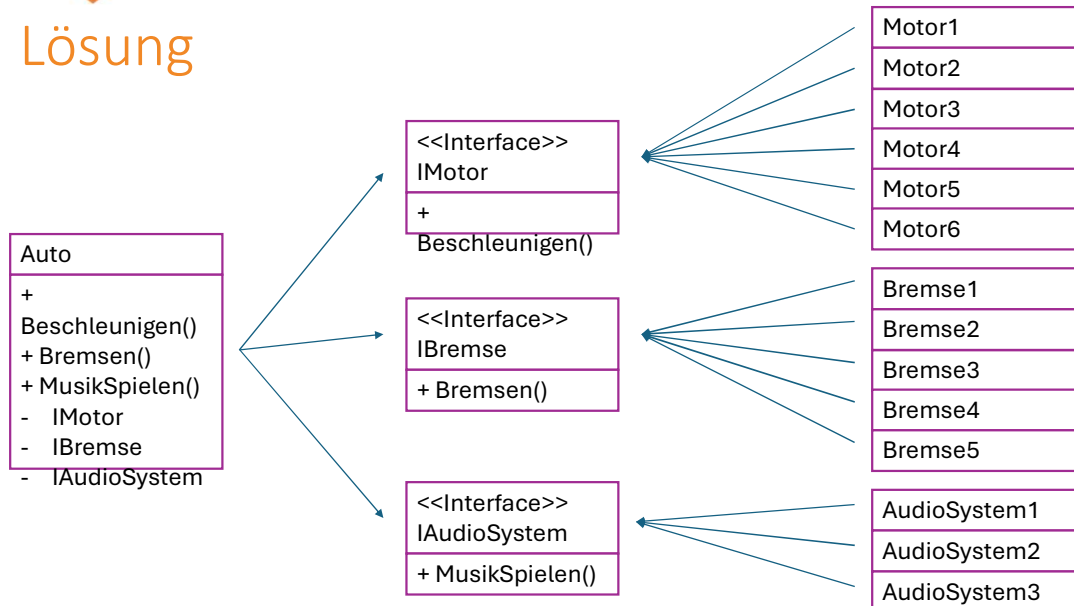
Problem mit Vererbung



Problem mit Vererbung

	Beschleunigen()		Bremsen()		MusikSpielen()		Alle Möglichkeiten
Varianten:	6	*	5	*	3	=	90
Änderung:	1 ändern	*	5	*	3	=	15 ändern
Erweitern:	7	*	5	*	3	=	105

Lösung



Vorteile

- Algorithmus kann zur Laufzeit festgelegt werden
- Algorithmus kann zur Laufzeit ausgetauscht werden

Nachteile

- Mehr Klasseninstanzen
- Mehr Schnittstellen
- Objekterzeugung Aufwändiger

Mehr Klasseninstanzen -> Jedes Instanz von Auto benötigt zusätzlich noch eine Instanz von Motor, von Bremse und eine von AudioSystem.

Iterator

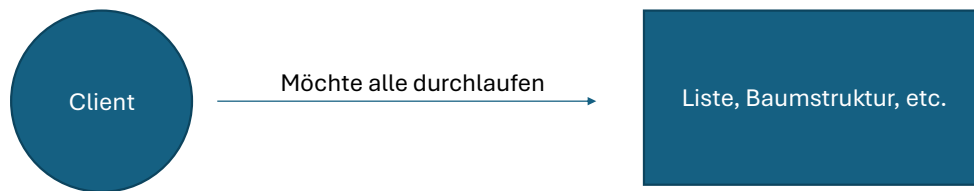
Behavioral Pattern

[https://de.wikipedia.org/wiki/Iterator_\(Entwurfsmuster\)](https://de.wikipedia.org/wiki/Iterator_(Entwurfsmuster))

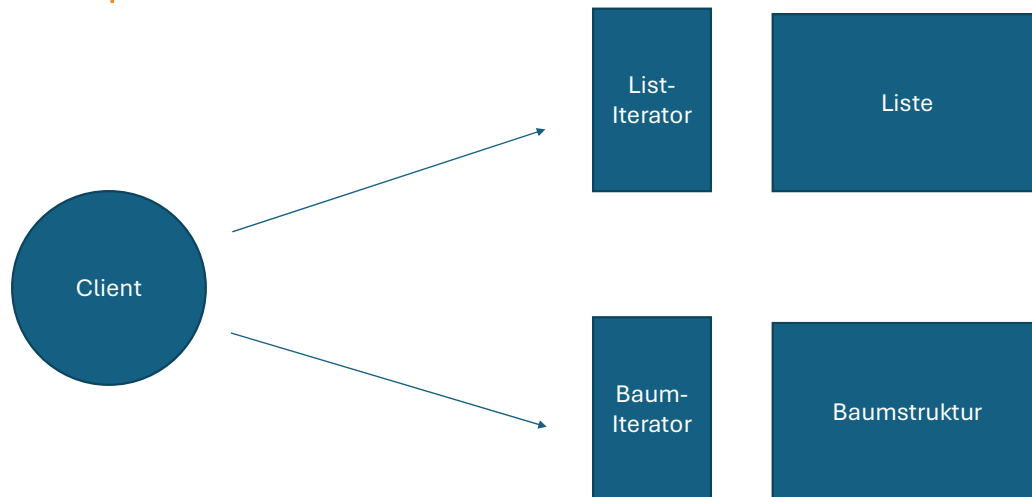
Verwendung

- Durchlaufen aller Teilelemente eines Objekts (Liste)
- IEnumerable

Beispiel



Beispiel



Vergleich ähnlicher Muster

Adapter vs. Fassade

- Beide: können ein oder mehrere Objekte umschleiern
- Adapter: Bietet einen Adapter von einer Schnittstelle zu einer anderen.



Adapter Drehstrom auf Gardena

Wenn man dringend Wasser braucht und nur einen klassischen Drehstromanschluss zur Verfügung hat genau das richtige

19.90 €

 In den Korb

- Fassade: Vereinfacht den zugriff auf eine komplexe Schnittstelle.

Builder vs. FactoryMethod

- Beide
 - Kapseln die Objekterzeugung
 - Entkoppeln den Client vom konkreten Objekttyp.
 - Mögliche Validierung vor Objekterzeugung.
- Builder: Behandelt den schrittweisen Erzeugungsprozess komplexer Objekte.
- Factory Method: Liefert ein fertiges Objekt mit einem Methodenaufruf.